PROGRAMACIÓN II / HITO 3

H3

NEXUS

Manual técnico y preparación para el examen H3

Academia de Exploradores · C# / .NET 8

Luis Mario Medrano Zeballos

Ingeniería en Sistemas

Octubre de 2026

Guía de lectura y evidencia

Archivos revisados: NexusTraining/Program.cs, NexusTraining/Simulacion.cs, NexusTraining/Pruebas.cs y README.md. Se mantiene el programa funcional; este manual explica el diseño real y amplía los temas que el examen exige estudiar. Mapa del contenido: I. Arquitectura y POO; II. Clases y objetos; III. Herencia, polimorfismo e interfaces; IV. Colecciones y LINQ; V. Análisis del funcionamiento; VI. Preguntas del examen; VII. Diagnóstico personal (prompt espejo).

I. Arquitectura y programación orientada a objetos

NEXUS es un simulador de consola en C#/.NET 8 ambientado en el año 2297. Registra a un cadete, valida la inmersión y permite explorar, decodificar IRIS, consultar estado, recargar, recalibrar y cerrar sesión. El estado de energía y estabilidad condiciona las decisiones. Responsabilidades reales Program: punto de entrada Main, interacción con consola, menú y suscripción al evento IRIS. Entrada: valida textos y enteros. Simulacion: modelo de estado y reglas del juego. Pruebas: ejecuta 12 escenarios automáticos sin paquetes externos. Diagrama de clases simplificado (basado en código real) Program (static) --crea--> Simulacion | | +--usa--> Entrada +-- propiedades: Energia, Estabilidad, Ecos... +--suscribe evento-------->+-- evento: AnomaliaIris +-- Operar(), Estado(), EvaluarSeguridad() Pruebas (static) --instancia y verifica--> Simulacion / Entrada La relación entre Program y Simulacion es de uso y creación, no de herencia. No existe una jerarquía de clases de juego en el código actual. Estructurada frente a POO En programación estructurada, una secuencia de funciones podría recibir energía y estabilidad como parámetros y devolver valores nuevos. En POO, Simulacion reúne datos y comportamientos en un objeto que mantiene sus invariantes. El menú de Program sigue un flujo estructurado, pero delega las reglas al objeto.

II. Fundamentos de clases y objetos

Clase, objeto, instancia, campos, propiedades y métodos La clase Simulacion define un tipo. new Simulacion(...) crea un objeto concreto. Las propiedades Energia y Estabilidad exponen lectura pública y escritura privada; el campo exploracionesSeguidas es interno. Los métodos Operar y Estado representan comportamiento.

```
var sesion = new Simulacion("Luis", 20, "LUMEN-07", 80, 80);
Console.WriteLine(sesion.Energia); // 80
```

sesion.Operar(1);

```
Console.WriteLine(sesion.Energia); // 68
```

Constructores y sobrecarga

El constructor real de Simulacion recibe cinco parámetros y comprueba sus rangos. No hay sobrecarga de constructores en esta clase. Ejemplo complementario, independiente del código real:

```
class Cadete
{
    public string Nombre { get; }
    public Cadete() : this("Sin nombre") { }
    public Cadete(string nombre) { Nombre = nombre; }
```

```
}
```

Dos constructores con firmas distintas constituyen sobrecarga. El segundo recibe el nombre; el primero delega mediante this(...).

Encapsulamiento, accesos, this y base

El código real usa internal sealed class Simulacion, miembros public, propiedades con private set y un método privado EvaluarSeguridad. Esto impide que el menú asigne arbitrariamente sesion.Energia = -100. this y base no son necesarios en su implementación actual; base se ilustra en la sección III.

III. Herencia, polimorfismo e interfaces

Nota: las siguientes clases son ejemplos complementarios de estudio; no forman parte del código

publicado. La clase real Simulacion es sealed, por lo que no puede utilizarse como clase base. Clase base, derivada, virtual, override, abstract y base abstract class Entidad

```
{
    public string Nombre { get; }
    protected Entidad(string nombre) { Nombre = nombre; }
    public abstract string Accion();
    public virtual string Presentarse() => "Entidad: " + Nombre;
}
class Explorador : Entidad
{
    public Explorador(string nombre) : base(nombre) { }
    public override string Accion() => "Explorar ecos";
    public override string Presentarse() => "Cadete: " + Nombre;
}
```

Entidad e = new Explorador("Luis");

```
Console.WriteLine(e.Accion()); // Explorar ecos
```

Entidad es abstracta y no se instancia directamente; Accion es abstracto y obliga a implementarlo. Presentarse es virtual y puede redefinirse. override permite que la llamada sobre una referencia Entidad ejecute la versión de Explorador: polimorfismo dinámico. base(nombre) invoca el constructor de la clase padre. Interfaces y contratos interface IExplorable { string Explorar(); }

```
class ZonaIris : IExplorable
{
    public string Explorar() => "Se encontró una señal IRIS";
}
```

IExplorable zona = new ZonaIris();

```
Console.WriteLine(zona.Explorar());
```

La interfaz define qué debe ofrecer el objeto, no la implementación concreta. Herencia expresa una relación «es un»; composición una relación «tiene un»; implementar una interfaz indica que se cumple un contrato. Por ejemplo, una Simulacion podría contener una lista de zonas (composición) sin heredar de ellas.

IV. Colecciones y LINQ

Colecciones que sí están en NEXUS

En el constructor de Simulacion se crea una List<string> llamada causas para reunir motivos de rechazo, y se agregan mensajes con Add. En Pruebas se usan arreglos, foreach, lambdas y delegados Action para ejecutar verificaciones.

```
var causas = new List<string>();
if (edad < 16) causas.Add("edad menor de 16");
if (energia < 40) causas.Add("energía menor de 40");
```

string motivo = string.Join("; ", causas);

LINQ y expresiones lambda: ejemplo complementario

No hay consultas LINQ Where, Select ni OrderBy en los tres archivos C# revisados. Para estudiar esas operaciones, este ejemplo usa una colección de exploradores ficticios:

```
using System.Linq;
var cadetes = new List<(string Nombre, int Energia)>
{ ("Ana", 75), ("Luis", 40), ("Eva", 90) };
var nombres = cadetes.Where(c => c.Energia >= 60)
    .OrderBy(c => c.Nombre)
    .Select(c => c.Nombre).ToList();
Console.WriteLine(string.Join(", ", nombres)); // Ana, Eva
```

Where filtra, OrderBy ordena, Select transforma y ToList materializa los resultados. Count puede contar elementos; Any comprobar si existe alguno. Una lambda como c => c.Energia >= 60 representa una función breve que recibe c y devuelve una condición. LINQ frente a foreach Con foreach se crea una lista de resultados, se recorre cada cadete, se evalúa un if y después se ordena. LINQ expresa las operaciones encadenadas de forma declarativa. Ambas opciones son válidas; LINQ suele ser más legible para consultas y foreach puede ser más claro cuando hay lógica de actualización con efectos laterales.

V. Análisis del funcionamiento

Flujograma de una sesión

```
INICIO
  |
  v
```

Leer nombre, edad, realidad, energia, estabilidad | v Crear Simulacion --> ¿Autorizada? --NO--> Mostrar motivo --> FIN | SI v Mostrar menú 1..6 | v Operar(opcion) | v Evaluar estado/seguridad | ¿Conectada sigue true? | SI | NO +-- volver |--> Mostrar resumen --> FIN

Seguimiento de un objeto real

Se crea Simulacion con edad 20, energía 80, estabilidad 80. Operar(1) consume 12 y 8: queda 68/72, Ecos=1. Operar(2) consume 18 y 12: queda 50/60, FragmentosIris=1. Operar(3) solo consulta, no cambia el estado. Operar(4) recupera 22 y resta 4: queda 72/56, Celdas=2. Operar(6) consume 10 y suma 18: queda 62/74.

Predicción de código y modificación

```
var s = new Simulacion("Luis", 20, "LUMEN-07", 80, 80);
```

s.Operar(1); s.Operar(3); s.Operar(1);

```
Console.WriteLine($"{s.Energia}/{s.Estabilidad}");
```

// Resultado: 49/58 Razón: la primera exploración cuesta 12/8. Consultar no reinicia exploracionesSeguidas. La segunda cuesta 19/14. Si se cambiara el caso 3 para reiniciar ese contador, la segunda exploración costaría 12/8 y el resultado sería 56/64. Esa modificación reduciría la penalización y alteraría el diseño del juego.

VI. Preparación para el examen — seis preguntas

razonadas

Pregunta: 1. ¿Por qué Simulacion es un ejemplo de POO y no solo un

conjunto de funciones?

Respuesta razonada: Porque representa una sesión concreta mediante propiedades y reglas

propias; el objeto conserva su estado entre llamadas. Program coordina la interacción, mientras Simulacion protege los cambios de energía y estabilidad. Esa separación favorece pruebas y mantenimiento.

Pregunta: 2. ¿Qué sucede al crear dos objetos Simulacion y ejecutar

Operar(1) solo en uno?

Respuesta razonada: Cada instancia tiene sus propios valores de Energia, Estabilidad y Ecos. Si

ambas comienzan en 80/80, solo la primera pasa a 68/72. Las propiedades privadas impiden cambiar estos valores libremente desde Program. El constructor valida que los estados iniciales sean coherentes.

Pregunta: 3. Si una variable Entidad contiene un Explorador, ¿qué método

se ejecuta al llamar Accion()?

Respuesta razonada: Se ejecuta Explorador.Accion() y devuelve «Explorar ecos», aunque la variable

sea de tipo Entidad. El método abstracto se implementa en la clase derivada y la resolución polimórfica depende del objeto real. Es un ejemplo complementario; Simulacion no hereda de Entidad.

Pregunta: 4. ¿Por qué usar IExplorable en vez de obligar a todas las zonas a

heredar de una sola clase?

Respuesta razonada: La interfaz exige una operación común sin imponer una jerarquía única. Una

zona podría heredar de otra clase y aun así implementar IExplorable. Facilita intercambiar implementaciones y probar componentes; herencia, composición e interfaz resuelven necesidades diferentes.

Pregunta: 5. Con los cadetes Ana 75, Luis 40 y Eva 90, ¿qué devuelve

Where(Energia >= 60).OrderBy(Nombre).Select(Nombre)?

Respuesta razonada: Devuelve Ana y Eva, en ese orden. Where excluye a Luis por su energía;

OrderBy ordena alfabéticamente los restantes; Select conserva solo sus nombres. LINQ no modifica la lista original y el resultado puede materializarse con ToList.

Pregunta: 6. Predice el estado tras Operar(1), Operar(3), Operar(1) desde

80/80 y explica cómo cambiaría al reiniciar el contador en la consulta.

Respuesta razonada: El resultado real es 49/58, porque consultar no rompe la secuencia de

exploraciones. Si la consulta reiniciara el contador, quedaría 56/64. Para modificarlo se debería cambiar la rama case 3 de Operar y actualizar las pruebas automatizadas para reflejar la nueva regla. Referencias y ejecución Repositorio: https://github.com/cbbeluismariomedranoze-jpg/NEXUS_Entrenamiento dotnet run --project NexusTraining dotnet run --project NexusTraining -- --pruebas La documentación original informa 12 pruebas aprobadas y 0 fallidas. Este manual es un análisis del código recuperado; no representa una nueva ejecución de esas pruebas. Fuentes conceptuales complementarias: Microsoft Learn, documentación de C# (clases, herencia, interfaces y LINQ).

VII. Resultado del PROMPT ESPEJO — diagnóstico

personal Alcance: diagnóstico basado en las interacciones disponibles de esta misión y el trabajo conversado sobre NEXUS. No se atribuye al estudiante el contenido que escribió la IA. Las confirmaciones de pruebas y las preguntas de seguimiento son evidencia de participación, no demostración automática de dominio conceptual.

1. Fortalezas demostradas

El estudiante identificó la necesidad de cotejar la entrega con la consigna real, compartió los requisitos completos y pidió verificar el contenido del PDF antes de entregarlo. Confirmó la ejecución de las seis operaciones y participó en el flujo de publicación del proyecto. Estas acciones evidencian seguimiento de requisitos, verificación práctica y disposición a corregir entregables; no demuestran por sí mismas dominio de herencia o LINQ.

2. Falencias observadas

Se confundieron inicialmente las seis operaciones probadas con las seis preguntas académicas de la sección VI; la distinción quedó aclarada al compartir el texto del docente. En los mensajes disponibles hay pocas explicaciones autónomas del estudiante sobre el comportamiento interno de clases, métodos, polimorfismo o consultas LINQ. Esto constituye evidencia insuficiente, no prueba de incapacidad.

3. Diagnóstico del Hito 3

POO: Comprensión parcial: seguimiento del proyecto y preguntas de verificación, sin explicación técnica propia suficiente. Clases y objetos: Evidencia insuficiente: no consta una explicación autónoma de instanciación o estado. Constructores: Evidencia insuficiente: no consta justificación propia de validación o sobrecarga. Encapsulamiento: Evidencia insuficiente: no consta análisis personal de private set y campos privados. Herencia: Evidencia insuficiente: no consta ejemplo razonado propio. Polimorfismo: Evidencia insuficiente: no consta predicción propia de despacho virtual. Interfaces: Evidencia insuficiente: no consta diseño de contrato explicado por el estudiante. Listas genéricas: Evidencia insuficiente: no consta análisis propio de List. LINQ: Evidencia insuficiente: no consta consulta propia ni interpretación de resultados.

4. Mi forma de aprender

El proceso observado es práctico e iterativo: probar, pedir revisión, comparar con la consigna y corregir la documentación. Una mejora sería explicar primero con palabras propias qué hará un fragmento y después ejecutarlo, anotando si la predicción fue correcta. Así se diferencia comprender del mero funcionamiento.

5. Plan de estudio prioritario

1. Clases y encapsulamiento: Dibujar Simulacion y explicar por qué Energia tiene private set. 2. Constructores y sobrecarga: Escribir una clase Cadete con dos constructores y justificar this(...). 3. Herencia y polimorfismo: Crear Entidad y Explorador; predecir el resultado de una llamada override. 4. Interfaces y composición: Definir IExplorable y comparar su uso con contener objetos ZonaIris. 5. Listas y LINQ: Crear una lista de cadetes, filtrar con Where y ordenar con OrderBy; repetir con foreach.

6. Conclusión

Se observan constancia en la verificación funcional y atención a los requisitos de entrega. Falta evidencia suficiente para certificar comprensión autónoma de los principales conceptos de POO avanzada y LINQ. Antes del examen, el estudiante debería poder describir la responsabilidad de cada archivo, predecir cambios de estado, justificar el uso de encapsulamiento, explicar override e interfaces y escribir una consulta LINQ sin ayuda. Nota metodológica: este diagnóstico responde a los seis apartados del prompt espejo suministrado por el docente y respeta su instrucción de no inventar evidencias ni atribuir al estudiante respuestas generadas por la IA.